

LẬP TRÌNH GAME CỜ CARO

NHÓM 6



NHIỆM VỤ HOÀN THÀNH

Thành viên	Nhiệm vụ	Hoàn thành
NTS Nguyễn Thái Sơn	Thiết kế TCP cơ bản · Gửi/nhận DTO & raise event · Xử lý lỗi mạng · Fix bug & Demo	100%
HTP Hồ Thiên Phúc	Dựng server & định nghĩa gói tin · Logic ghép trận & đồng bộ thời gian · Xây dựng AI · Fix lỗi	100%
DHT Đặng Hồng Tân	Set up Form Client & bàn Caro · Form gameplay & kết nối Network · Chat Form + Timer · Fix bug	100%
NMD Nguyễn Minh Đăng	Forms (Server Dashboard UI) · Models + DTOs + Services · Database migration · Integration & Testing	100%
TNT Trần Nguyễn Tân Tài	Xử lý tương tác & vẽ bàn cờ · Ghép hệ thống & tối ưu · Bẫy lỗi bất đồng bộ · Hỗ trợ Network	100%

THIẾT KẾ NETWORK

1. KIẾN TRÚC TỔNG THỂ NETWORK

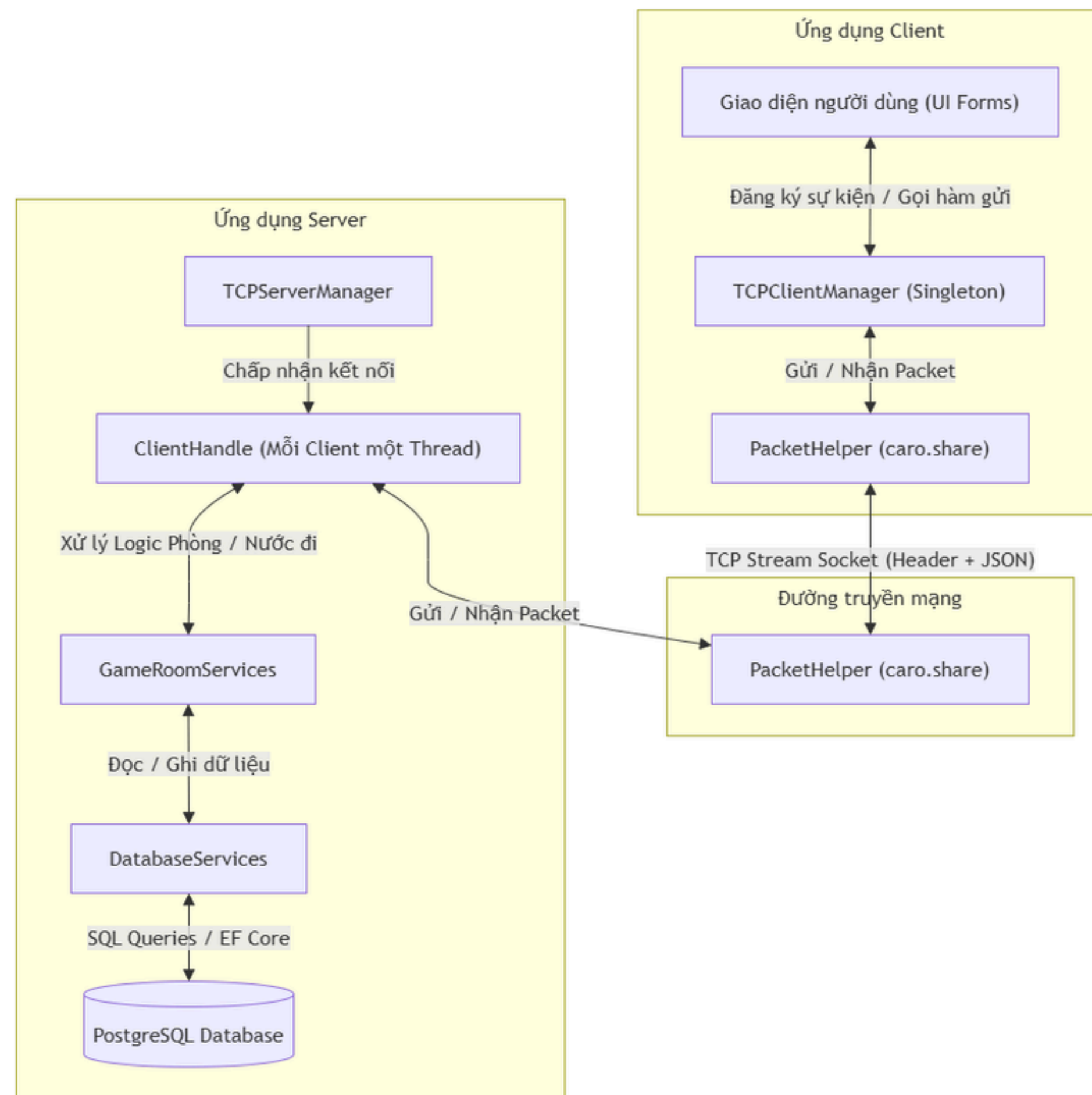
2. QUY TRÌNH GỬI GÓI TIN

3. QUY TRÌNH NHẬN VÀ XỬ LÝ GÓI TIN



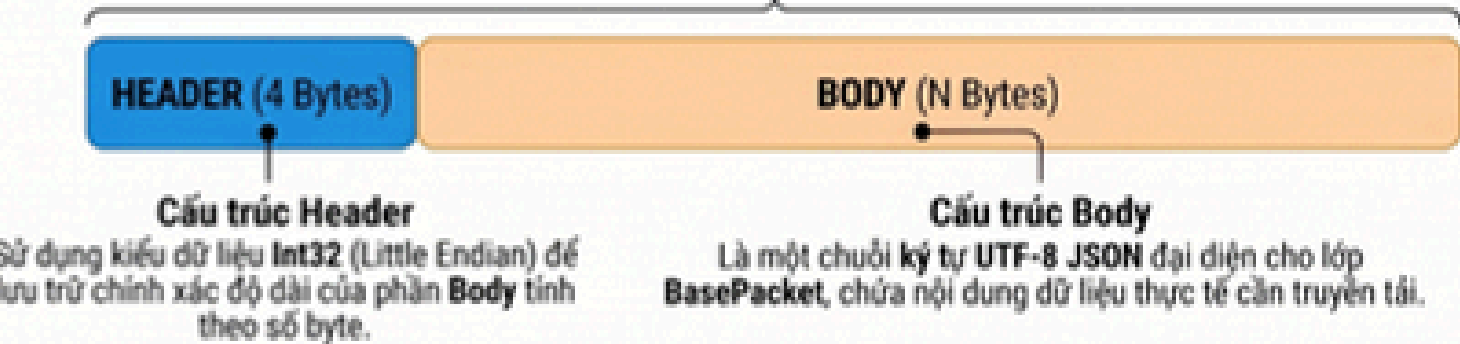
NETWORKS

KIẾN TRÚC TỔNG THỂ NETWORK



GIAO THỨC TỰ ĐỊNH NGHĨA & DTO

Cơ chế Đóng khung Gói tin (Packet Framing)

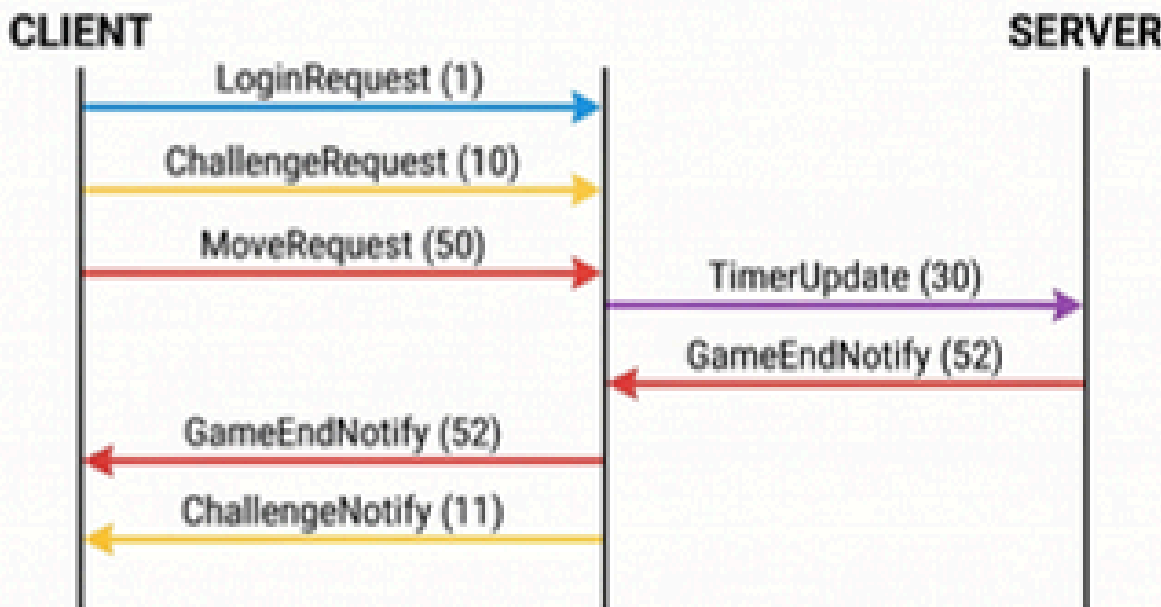


Quy trình xử lý
tại máy nhận

Đọc 4 byte đầu tiên
để xác định độ dài N

Đọc đúng N byte tiếp theo
để trích xuất nội dung Body mà
không sợ bị lẫn với gói tin khác.

Lưu thông Giao tiếp Hệ thống (Client <-> Server Flow)



Yêu cầu từ Client: Client gửi các gói tin như LoginRequest, ChallengeRequest hoặc MoveRequest để bắt đầu một hành động.

Phản hồi & Điều phối từ Server: Server xử lý logic và gửi lại kết quả (Response) hoặc thông báo (Notify) cho các bên liên quan, ví dụ: TimerUpdate hoặc GameEndNotify.

Auth Thách đấu Trận đấu/Nước đi Trò chuyện Thời gian Thống kê

Danh mục Loại Gói tin & DTO Payload (Nhóm Nghiệp vụ & Điều khiển)

Auth	PacketType (Enum)	DTO Payload	Hướng truyền	Chức năng chính
	1 - LoginRequest	LoginRequestDTO	→	Đăng nhập hệ thống
	2 - LoginResponse	LoginResponseDTO	←	Trả về kết quả đăng nhập
Thách đấu	10 - ChallengeRequest	ChallengeRequestDTO	→	Gửi lời thách đấu
	11 - ChallengeNotify	ChallengeNotifyDTO	←	Thông báo có người thách đấu
Trò chuyện	PacketType (Enum)	DTO Payload	Hướng truyền	Chức năng chính
	20 - ChatSend	ChatSendDTO	→	Gửi tin nhắn
Thời gian	30 - TimerUpdate	TimerUpdateDTO	←	Cập nhật đồng hồ đếm ngược
Trận đấu/Nước đi	PacketType (Enum)	DTO Payload	Hướng truyền	Chức năng chính
	50 - MoveRequest	MoveRequestDTO	→	Gửi nước đi (tọa độ cờ)
	51 - MoveNotify	MoveNotifyDTO	←	Thông báo nước đi & chuyển lượt
	52 - GameEndNotify	GameEndNotifyDTO	←	Kết thúc trận & báo người thắng
Thống kê	71 - BestRecordResponse	BestRecordResponseDTO	←	Trả về bảng xếp hạng



QUY TRÌNH GỬI GÓI TIN



MỤC TIÊU: Đóng gói dữ liệu thành Packet chuẩn trước khi truyền qua TCP, đảm bảo Server có thể xác định chính xác ranh giới gói tin.



CẤU TRÚC GÓI TIN ĐƯỢC GỬI ĐI

=

HEADER (4 BYTES)

+

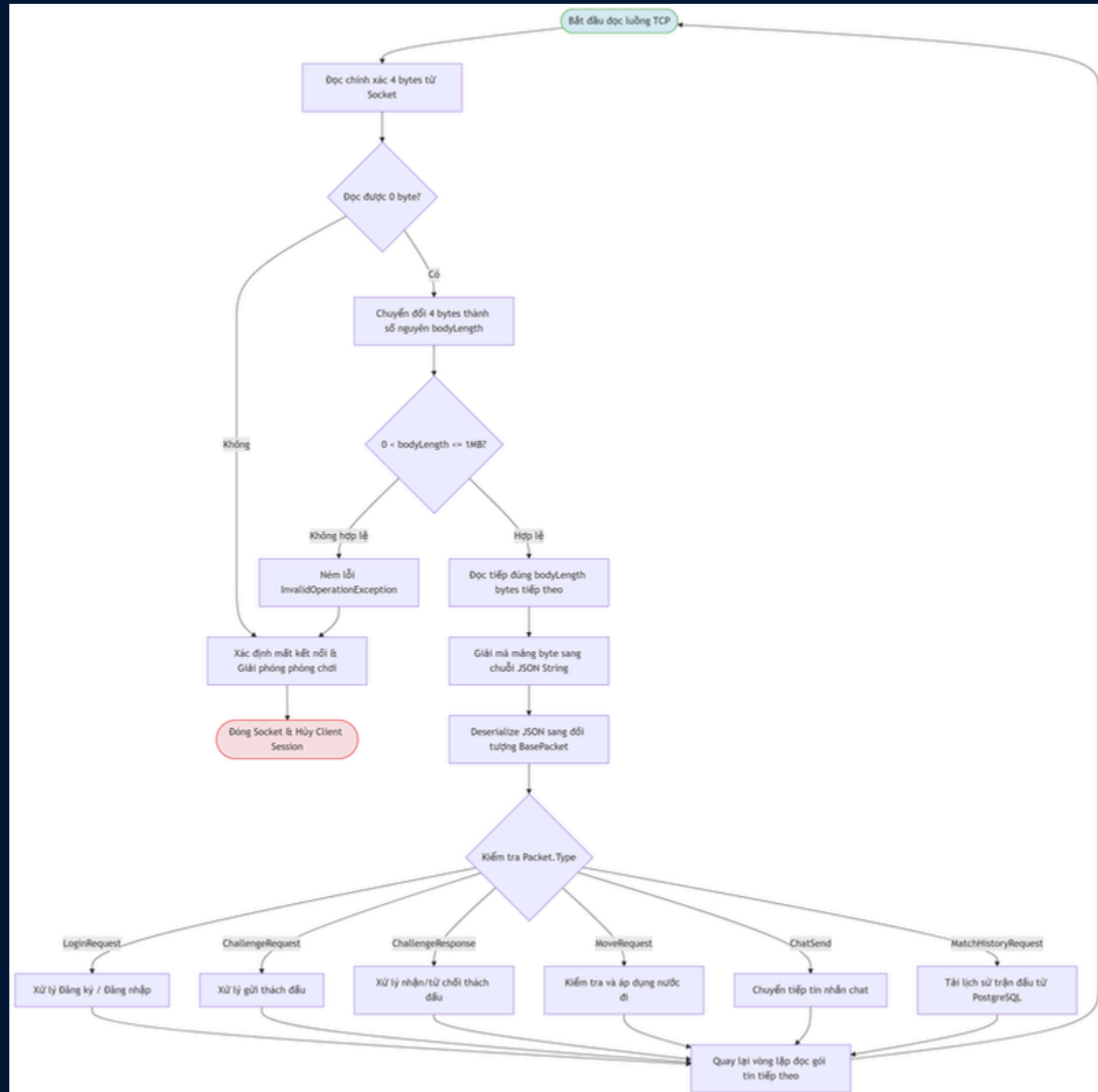
BODY (N BYTES)

HEADER: 4 bytes (độ dài N)

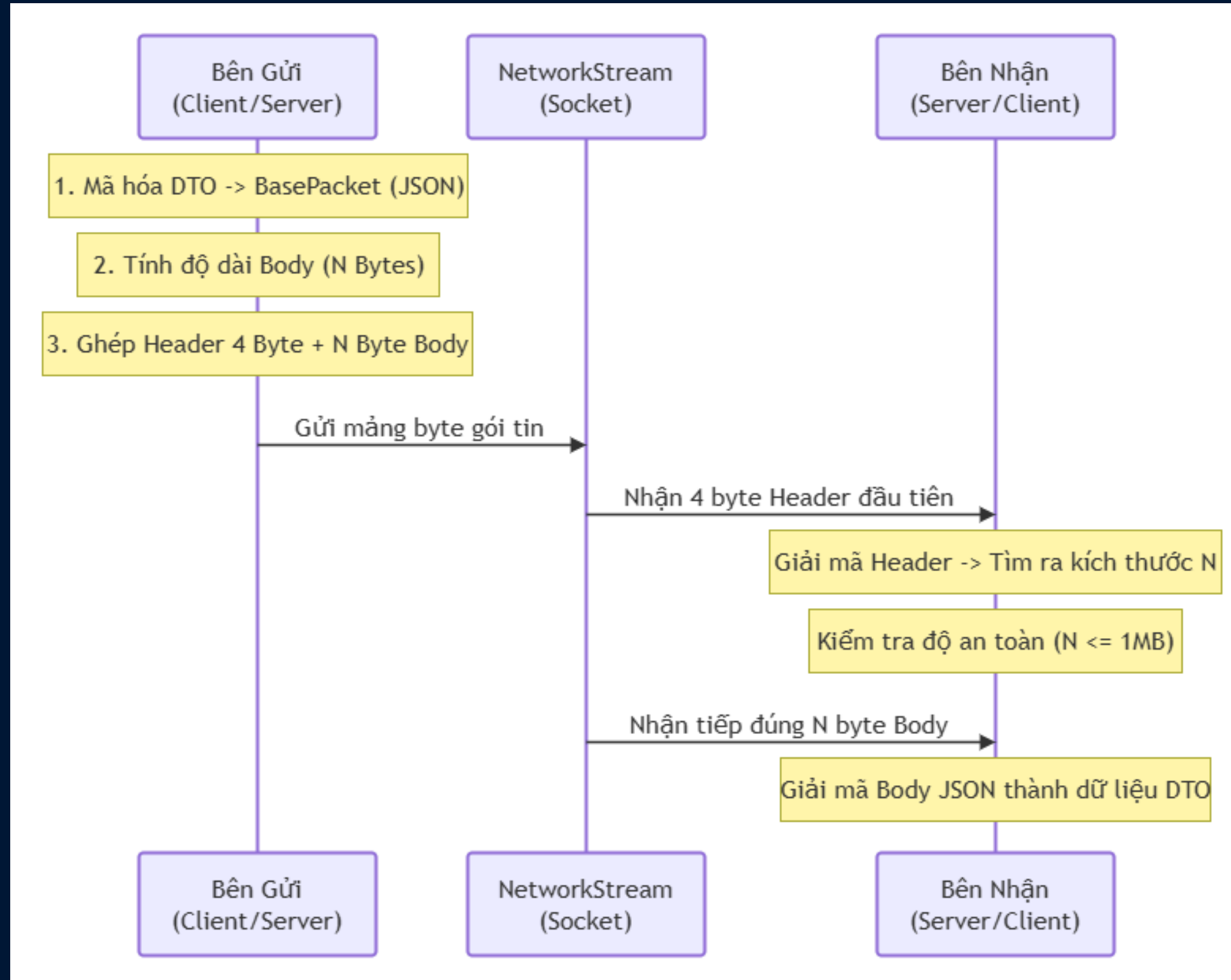
BODY: N bytes (dữ liệu JSON)

NETWORKS

QUY TRÌNH NHẬN VÀ XỬ LÍ GÓI TÍN



QUY TRÌNH TRUYỀN NHẬN DỮ LIỆU



NETWORKS



6 references

```
public class PacketHelper
```

```
{
```

4 references

```
public static async Task SendPacketAsync<T>(NetworkStream stream, T packet)
```

```
{
```

```
    // Mã hóa object thành json string
```

```
    string json = JsonSerializer.Serialize(packet);
```

```
    // Chuyển thành mảng các byte
```

```
    byte[] bodybyte = Encoding.UTF8.GetBytes(json);
```

```
    // Ép thành mảng 4 byte theo bodybyte.Length -> biết kích thước nội dung gói tin
```

```
    byte[] headerbyte = BitConverter.GetBytes(bodybyte.Length);
```

```
    // Ghép header với body
```

```
    byte[] fullpacket = new byte[headerbyte.Length + bodybyte.Length];
```

```
    Buffer.BlockCopy(headerbyte, 0, fullpacket, 0, headerbyte.Length);
```

```
    Buffer.BlockCopy(bodybyte, 0, fullpacket, headerbyte.Length, bodybyte.Length);
```

```
    await stream.WriteAsync(fullpacket, 0, fullpacket.Length);
```

```
}
```


NETWORK

2 references

```
public static async Task<T> ReceivePacketAsync<T>(NetworkStream stream)
{
    byte[] headerbuffer = await ReadfullpacketAsync(stream, 4);
    int bodylength = BitConverter.ToInt32(headerbuffer, 0);

    // Giới hạn kích thước gói tin nhận tối đa là 1MB để tránh tấn công DoS tràn RAM
    const int MAX_PACKET_SIZE = 1024 * 1024; // 1 MB
    if (bodylength <= 0 || bodylength > MAX_PACKET_SIZE)
    {
        throw new InvalidOperationException("Kích thước gói tin không hợp lệ hoặc quá lớn!!!");
    }

    byte[] bodybuffer = await ReadfullpacketAsync(stream, bodylength);
    string json = Encoding.UTF8.GetString(bodybuffer);
    var result = JsonSerializer.Deserialize<T>(json);
    if (result == null)
    {
        throw new InvalidOperationException("Không thể giải mã gói tin!!!");
    }
    return result;
}
```

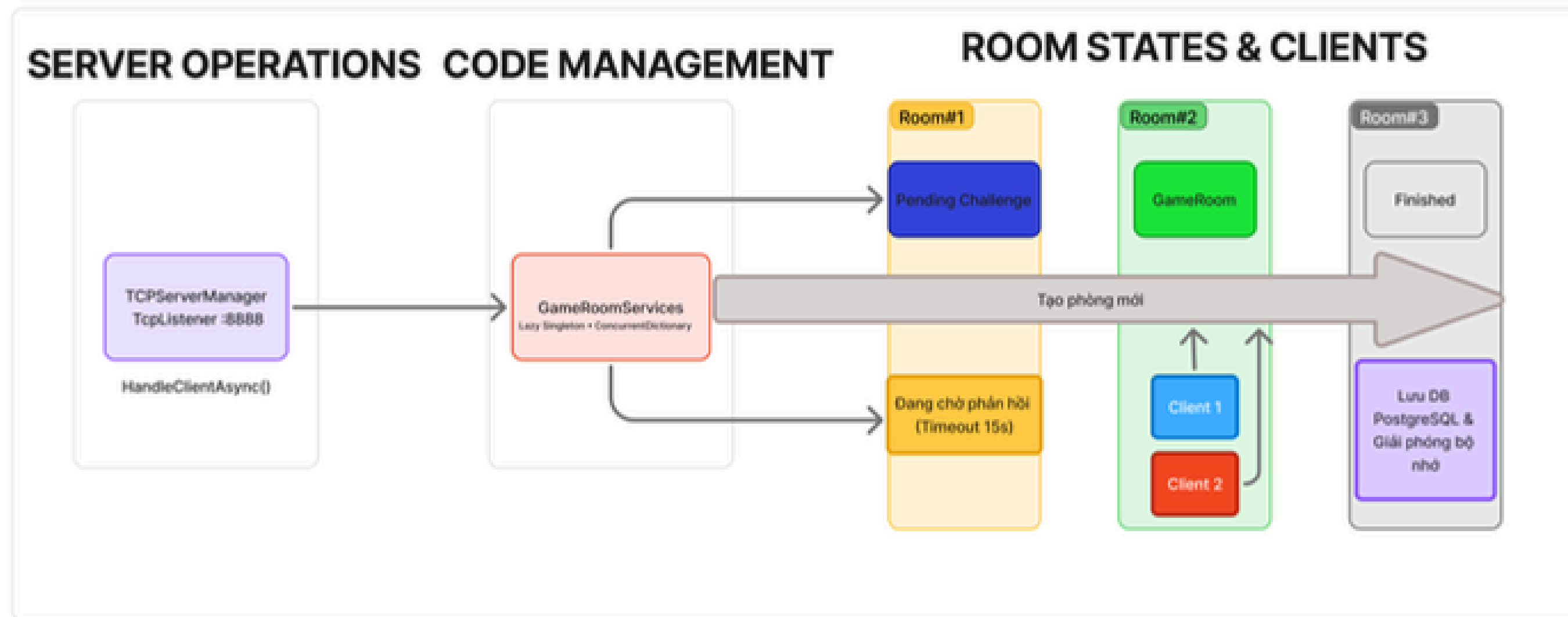

NETWORKS

2 references

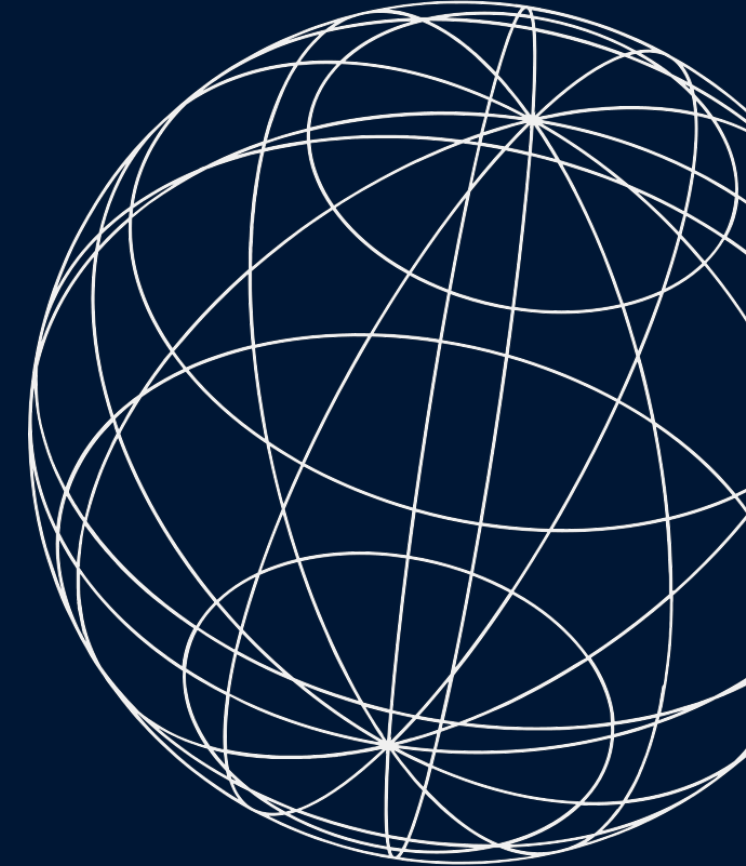
```
private static async Task<byte[]> ReadfullpacketAsync(NetworkStream stream, int count)
{
    byte[] buffer = new byte[count];
    int totalread = 0;
    while (totalread < count)
    {
        int byteread = await stream.ReadAsync(buffer, totalread, count - totalread);
        if (byteread == 0)
        {
            throw new SocketException((int)SocketError.ConnectionAborted);
        }
        totalread += byteread;
    }
    return buffer;
}
```



PHÂN TÍCH GAME ROOM



PHÂN TÍCH GAME ROOM



```
11 references
public class GameRoomServices
{
    private static readonly Lazy<GameRoomServices> _instance = new Lazy<GameRoomServices>(() => new GameRoomServices());

    6 references
    public static GameRoomServices Instance => _instance.Value;

    private static readonly ConcurrentDictionary<string, GameRoom> _activeroom = new();

    1 reference
    private GameRoomServices() { }

    2 references
    public async Task<GameRoom> CreateAndStartRoomAsync(ClientHandle player1, ClientHandle player2, int timesecons = 15)
    {
        var room = new GameRoom
        {
            RoomID = Guid.NewGuid().ToString("N").Substring(0, 8),
            player1 = player1,
            player2 = player2,
            TimeSecondPerPlayer = timesecons,
            RemainingTimeP1 = timesecons,
            RemainingTimeP2 = timesecons,
            CurrentTurn = player1.username, // mặc định đi trước
            IsGameActive = true,
            cts = new CancellationTokenSource()
        };
    }
}
```



PHÂN TÍCH GAME ROOM

```
player1.CurrentRoomID = room.RoomID;
player2.CurrentRoomID = room.RoomID;

_activeroom.TryAdd(room.RoomID, room);

var startnotify = new GameStartNotifyDTO
{
    roomid = room.RoomID,
    name_player1 = player1.username,
    name_player2 = player2.username,
    timeSeconds = timesecons
};
var packet = new BasePacket
{
    Type = PacketType.GameStartNotify,
    payload = JsonSerializer.Serialize(startnotify)
};
Task task1 = SendToPlayerAsync(player1, packet);
Task task2 = SendToPlayerAsync(player2, packet);

await Task.WhenAll(task1, task2);

TCPServerManager.Log($"[Dịch vụ Phòng] Trận đấu bắt đầu: '{player1.username}' VS '{player2.username}' (Phòng: {room.RoomID})")
_ = Task.Run(() => RunTimerLoopAsync(room, room.cts.Token));

if (room.CurrentTurn == "AI_Bot")
{
    _ = Task.Run(() => TriggerAIMove(room));
}
```



PHÂN TÍCH GAME ROOM



```
public void CleanupRoom(string RoomID, bool isPlayerWin = false, string username = "")
{
    if (_activeroom.TryRemove(RoomID, out var room))
    {
        room.IsGameActive = false;
        room.cts?.Cancel();
        if (!isPlayerWin)
        {
            string winnerName = "";
            if (room.player1 != null && string.Equals(room.player1.username, username, StringComparison.OrdinalIgnoreCase))
            {
                winnerName = room.player2?.username ?? "";
            }
            else if (room.player2 != null && string.Equals(room.player2.username, username, StringComparison.OrdinalIgnoreCase))
            {
                winnerName = room.player1?.username ?? "";
            }

            TCPServerManager.Log($"[Hệ thống phòng] Phát hiện ngắt kết nối đột ngột từ người chơi '{username}' trong phòng '{RoomID}'");

            var endNotify = new GameEndNotifyDTO
            {
                WinnerName = winnerName,
                reason = "Đối thủ đã mất kết nối đột ngột!"
            };
            var packet = new BasePacket
            {
                Type = PacketType.GameEnd,
                Data = endNotify
            };
            TCPServerManager.SendToAll(packet);
        }
    }
}
```



PHÂN TÍCH GAME ROOM



```
TCPServerManager.Log($"[Hệ thống phòng] Phát hiện người kết nối đột ngột cả người chơi {lastName} trong phòng {RoomID}");

var endNotify = new GameEndNotifyDTO
{
    WinnerName = winnerName,
    reason = "Đối thủ đã mất kết nối đột ngột!"
};
var packet = new BasePacket
{
    Type = PacketType.GameEndNotify,
    payload = JsonSerializer.Serialize(endNotify)
};

TCPServerManager.Log($"[Hệ thống phòng] Đang phát sóng gói tin kết thúc trận đấu (GameEndNotify) tới cả hai người chơi");
_ = SendToPlayerWithoutCleanupAsync(room.player1, packet);
_ = SendToPlayerWithoutCleanupAsync(room.player2, packet);
}

if (room.player1 != null) room.player1.CurrentRoomID = null;
if (room.player2 != null) room.player2.CurrentRoomID = null;

TCPServerManager.Log($"[Hệ thống phòng] Phòng đấu '{RoomID}' đã được dọn dẹp và giải phóng.");
}
else
{
    TCPServerManager.Log($"[Hệ thống phòng] [Cảnh báo] Không thể tìm thấy phòng đấu '{RoomID}' để giải phóng hoặc phòng đã đư
}
}
```



EXCEPTION HANDLING

Quy trình Xử lý Ngoại lệ (Exception Handling) trong Caro Server

Quy trình này mô tả cách Caro Server quản lý vòng đời của một kết nối TcpClient. Hệ thống tập trung vào việc giám sát các điểm dễ xảy ra lỗi bằng khối Try-Catch và thực hiện dọn dẹp dữ liệu triệt để khi có ngoại lệ phát sinh.



1. Tiếp nhận Kết nối TcpClient

TCPServerManager tiếp nhận yêu cầu và khởi tạo đối tượng ClientHandle để quản lý người chơi



2. Thiết lập Stream & Lắng nghe

Khởi tạo NetworkStream và bắt đầu vòng lặp (loop) lắng nghe dữ liệu liên tục từ phía Client



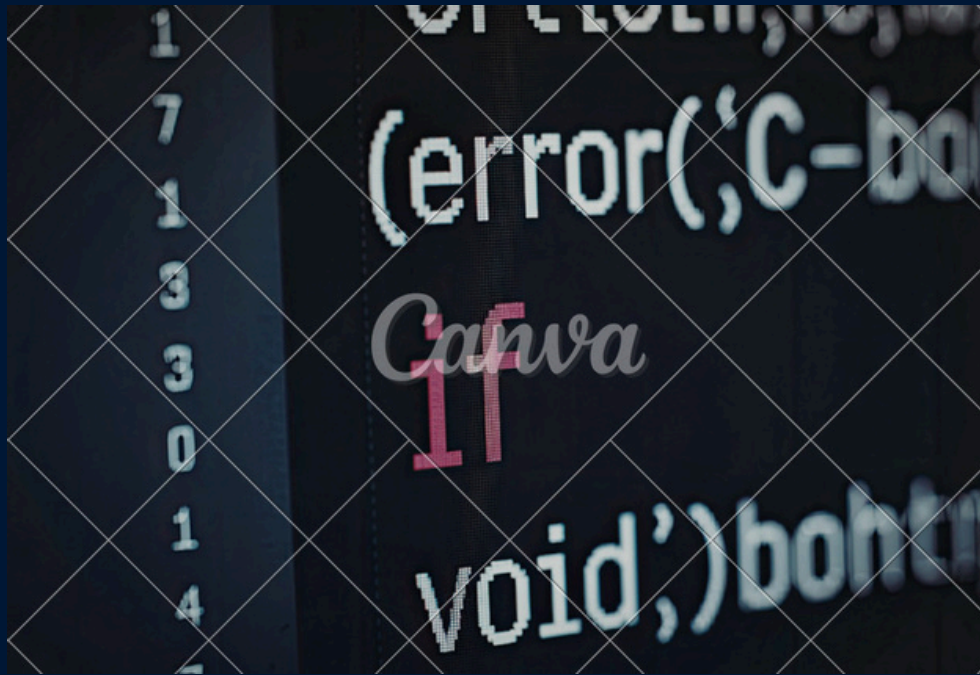
3. Giám sát & Bắt ngoại lệ (Try-Catch)

Áp dụng Try-Catch khi đọc dữ liệu stream, giải mã cấu trúc JSON và xử lý logic nghiệp vụ



4. Quy trình Dọn dẹp (Cleanup)

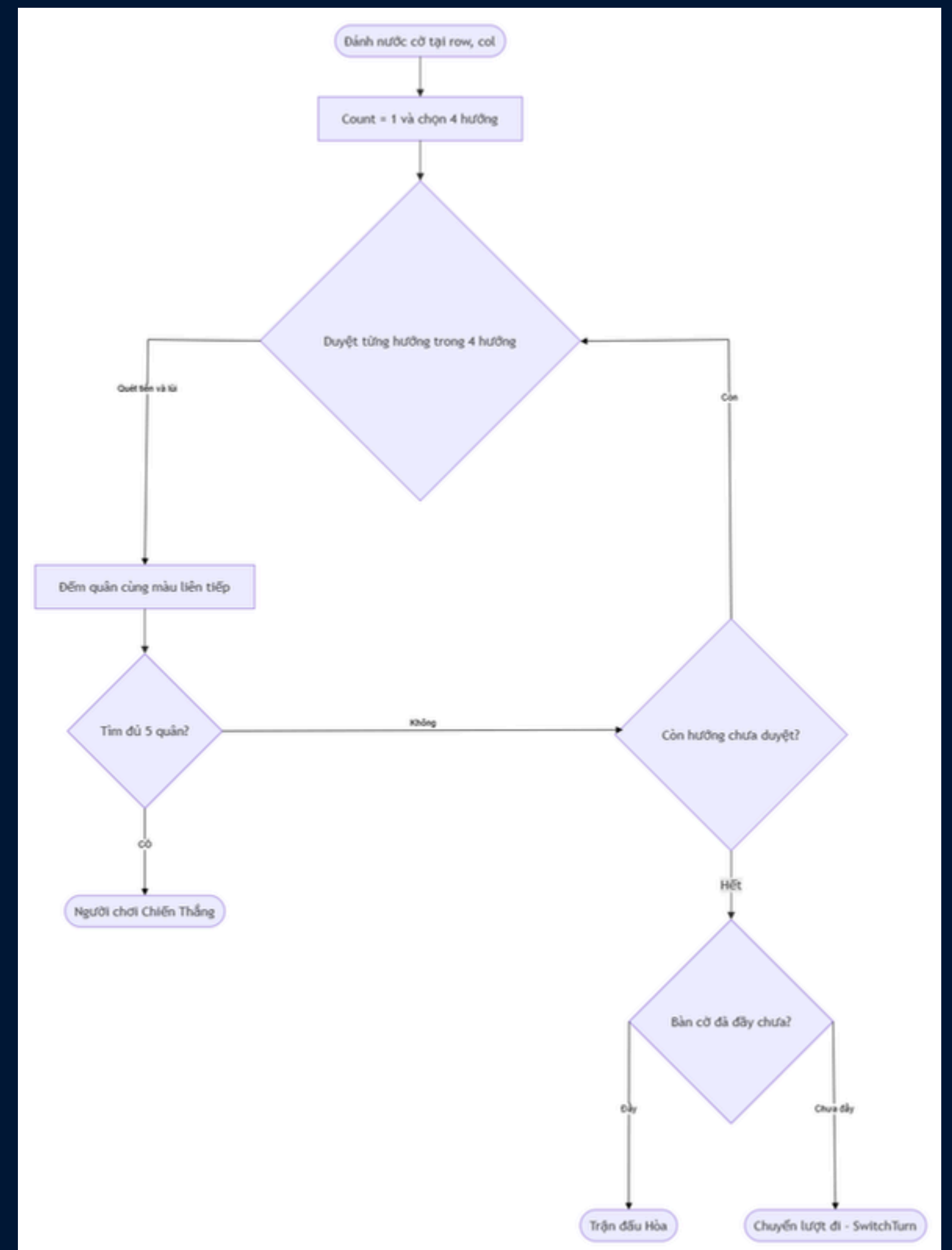
Đóng NetworkStream, ngắt kết nối TcpClient và xóa người chơi khỏi danh sách onlineplayer



ĐA LUỒNG & BẤT ĐỒNG BỘ

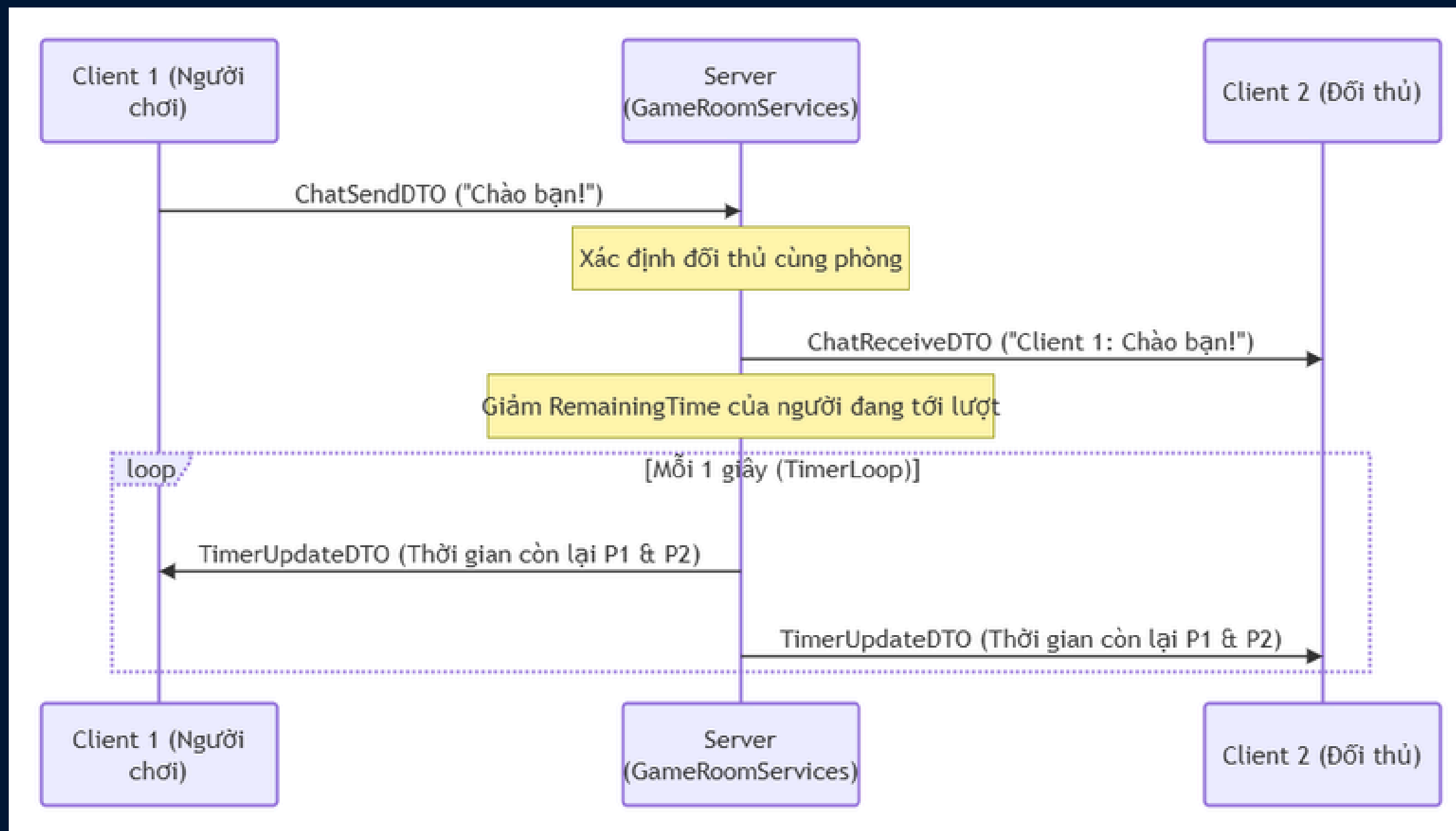


LOGIC CHECK WIN



CÁC TÍNH NĂNG NỔI BẬT

- CHAT & TIME



CÁC TÍNH NĂNG NỔI BẬT

- CHAT & TIME

```
1 reference
public async Task RunTimerLoopAsync(GameRoom room, CancellationToken ct)
{
    try
    {
        while (room.IsGameActive && !ct.IsCancellationRequested)
        {
            await Task.Delay(1000, ct);

            if (room.CurrentTurn == room.player1.username)
            {
                room.RemainingTimeP1--;
            }
            else
            {
                room.RemainingTimeP2--;
            }

            // Chuẩn bị payload
            var timeUpdate = new TimerUpdateDTO
            {
                RemainingTimePlayer1 = room.RemainingTimeP1,
                RemainingTimePlayer2 = room.RemainingTimeP2,
                CurrentTurnUserName = room.CurrentTurn
            };

            var packet = new BasePacket
            {
                Type = PacketType.TimerUpdate,
```



CÁC TÍNH NĂNG NỔI BẬT

- CHAT & TIME

```
{
    Type = PacketType.TimerUpdate,
    payload = JsonSerializer.Serialize(timeUpdate)
};
_ = SendToPlayerAsync(room.player1, packet);
_ = SendToPlayerAsync(room.player2, packet);

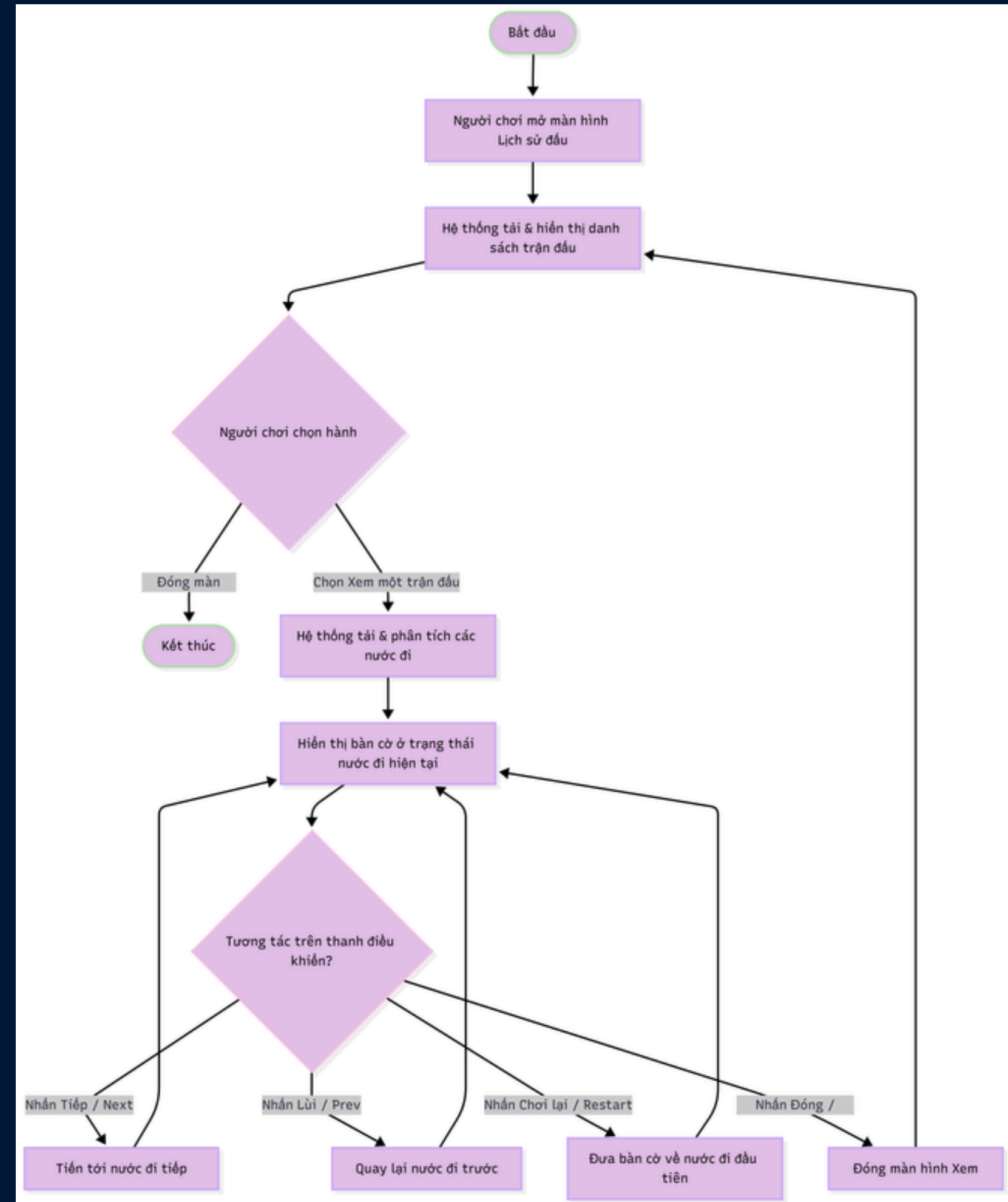
if (room.RemainingTimeP1 <= 0)
{
    await HandleTimerExpiredAsync(room, room.player1, room.player2);
    return;
}
else if (room.RemainingTimeP2 <= 0)
{
    await HandleTimerExpiredAsync(room, room.player2, room.player1);
    return;
}
}

}
catch (OperationCanceledException)
{
    TCPServerManager.Log($"[Trận đấu - Phòng {room.RoomID}] Đồng hồ đếm ngược đã dừng (trận đấu kết thúc hoặc dọn phòng).");
}
catch (Exception ex)
{
    TCPServerManager.Log($"[Trận đấu - Phòng {room.RoomID}] Lỗi đồng hồ đếm ngược: {ex.Message}");
}
}
```

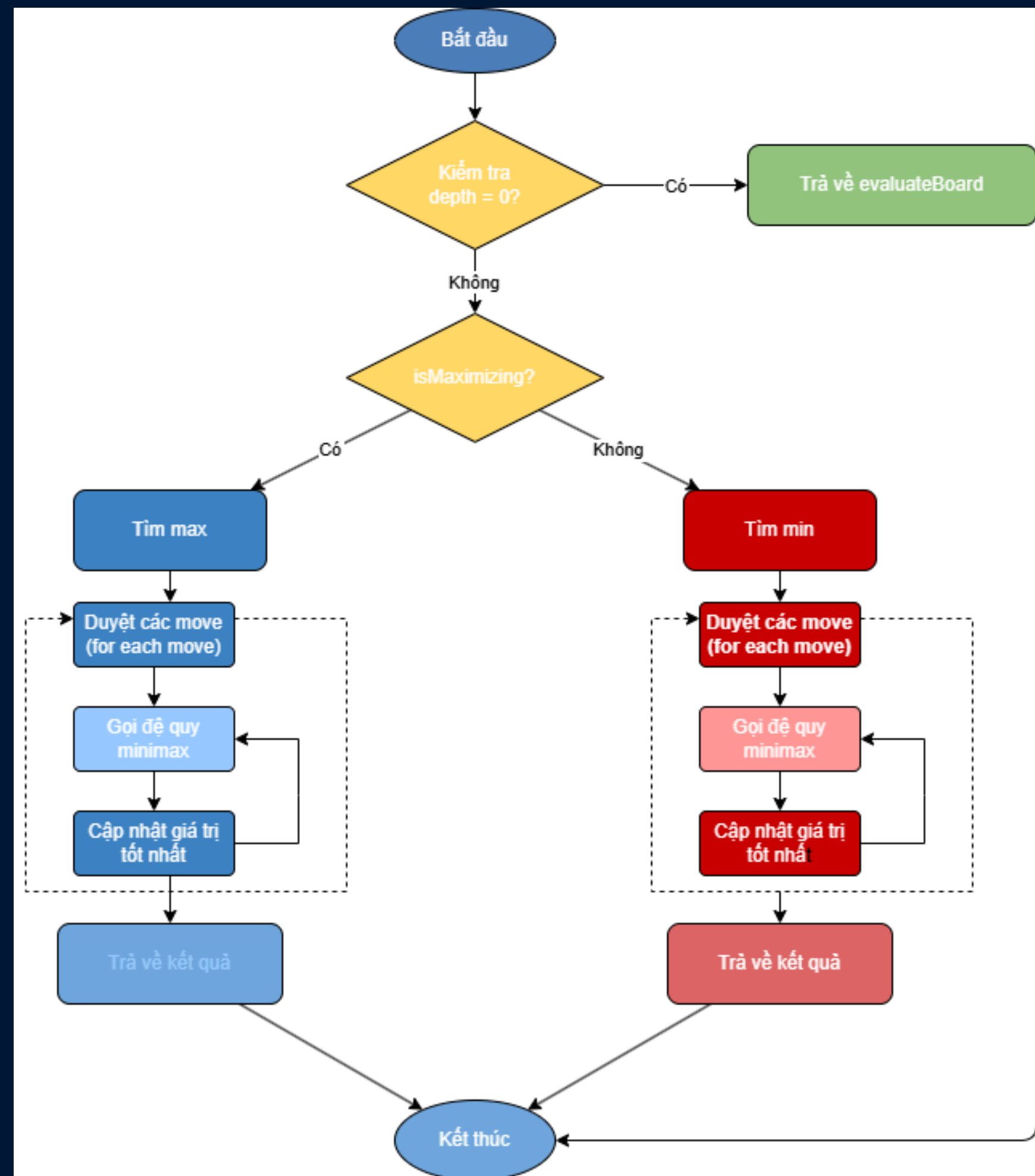


CÁC TÍNH NĂNG NỔI BẬT

- XEM LẠI LỊCH SỬ ĐẦU



THI ĐẤU VỚI AI



THI ĐẤU VỚI AI

3 references

```
public static (int score, (int r, int c)? move) MiniMax(int[,] boardstate, int depthLimit,
    int alpha, int beta, bool isMaximizing, int aiId)
{
    var status = CheckWin(boardstate);
    if (status.Winner != null || status.IsDraw || depthLimit == 0)
    {
        int score = EvaluateBoard(boardstate, aiId);
        if (status.Winner == aiId) score += 1000000 + depthLimit * 1000;
        else if (status.Winner != null) score -= 1000000 + depthLimit * 1000;
        return (score, null);
    }

    var validMoves = GetVaildMove(boardstate);

    if (validMoves.Count == 0) return (EvaluateBoard(boardstate, aiId), null);

    int currPlayer = isMaximizing ? aiId : (aiId == 1 ? 2 : 1);

    if (depthLimit >= 2)
    {
        var scoreMoves = new List<ScoredMove>();

        foreach (var move in validMoves)
        {
            boardstate[move.r, move.c] = currPlayer;
            int s = EvaluateBoard(boardstate, currPlayer);
            boardstate[move.r, move.c] = EMPTY;
            scoreMoves.Add(new ScoredMove { Move = move, Score = s });
        }
    }
}
```



THI ĐẤU VỚI AI

```
        scoreMoves.Sort((a, b) => b.Score.CompareTo(a.Score));
        validMoves = scoreMoves.Select(m => m.Move).ToList();
    }

    (int r, int c)? bestMove = null;

    if (isMaximizing)
    {
        int bestScore = int.MinValue;

        foreach (var move in validMoves)
        {
            boardstate[move.r, move.c] = currPlayer;
            var result = MiniMax(boardstate, depthLimit - 1, alpha, beta, false, aiId);
            boardstate[move.r, move.c] = EMPTY;

            if (result.score > bestScore)
            {
                bestScore = result.score;
                bestMove = move;
            }
            alpha = Math.Max(alpha, bestScore);
            if (alpha >= beta) break;
        }
        return (bestScore, bestMove);
    }
    else
    {
        int bestScore = int.MaxValue;
```

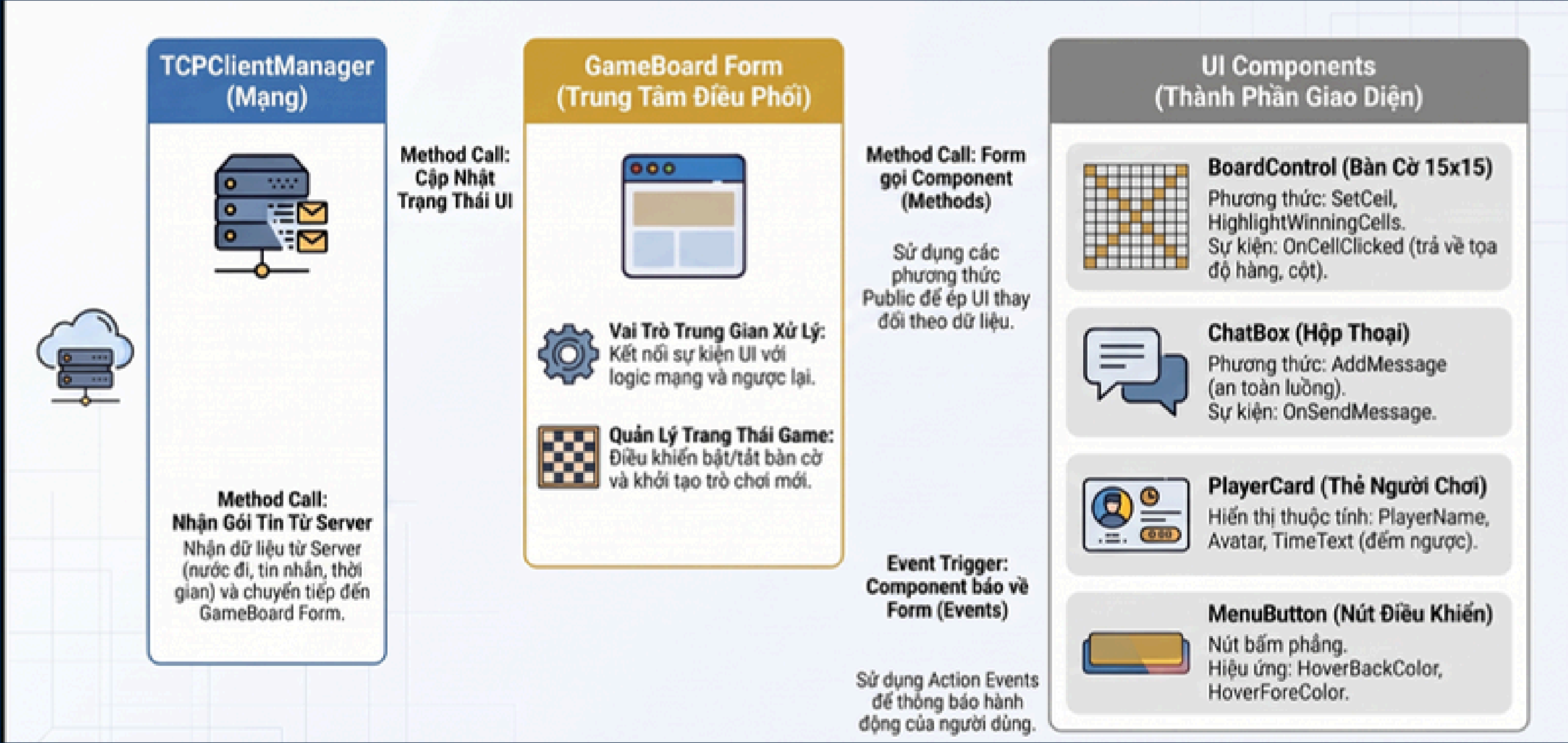


THI ĐẤU VỚI AI



```
foreach (var move in validMoves)
{
    boardstate[move.r, move.c] = currPlayer;
    var result = MiniMax(boardstate, depthLimit - 1, alpha, beta, true, aiId);
    boardstate[move.r, move.c] = EMPTY;
    if (result.score < bestScore)
    {
        bestScore = result.score;
        bestMove = move;
    }
    beta = Math.Min(beta, bestScore);
    if (beta <= alpha) break; // Cắt tia
}
return (bestScore, bestMove);
}
```

TƯƠNG TÁC FORM & UI

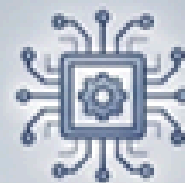


TƯƠNG TÁC SERVER & UI

Kiến Trúc Event-Driven: Luồng Tương Tác Giữa Server & UI Trong Game Caro

LUỒNG 1 - SERVER UI (HỆ THỐNG LOG & QUẢN TRỊ)

BACKGROUND THREAD

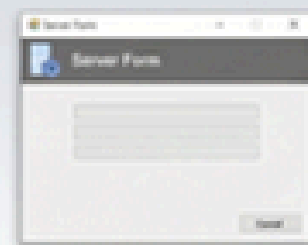


Logic Server Kích Hoạt



Sự Kiện Tĩnh (Static Events)

Khai báo tĩnh trong TCPServerManager
(OnLogMessage, OnPlayerConnectionChanged)



Đăng ký tại Server Form
(Home.cs)

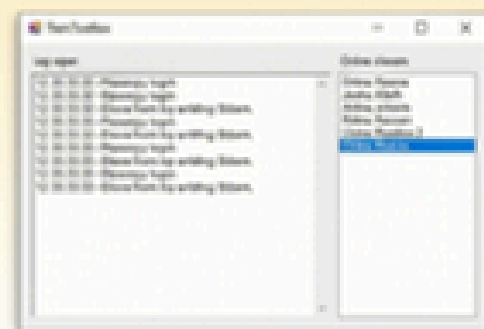
Form chính đăng ký ngay khi
khởi tạo để theo dõi biến động



Marshalling qua Invoke

Sử dụng control.Invoke để
chuyển tiếp đồng bộ về luồng
giao diện

UI THREAD



Cập nhật Log
& Danh sách

Ghi nội dung hoặc
cập nhật danh sách
người chơi online

LUỒNG 2 - CLIENT UI (GIAO DIỆN NGƯỜI CHƠI)

BACKGROUND THREAD



Nhận gói tin TCP (JSON)

Vòng lặp ReceiveLoopAsync
nhận dữ liệu trên luồng nền



Phân phối qua Instance Events

TcpClientManager (Singleton) giải
mã JSON và kích hoạt sự kiện



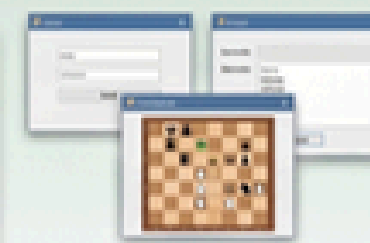
Form UI đăng ký sự kiện

Các Form đăng ký/hủy dựa trên
trạng thái hiển thị



Kiểm tra InvokeRequired

Đảm bảo Thread-safety trước
khi cập nhật UI



SƠ SÁNH CƠ CHẾ XỬ LÝ

Invoke
(Đồng bộ)



Chặn luồng nền
cho đến khi UI
xong việc
(e.g., Log, Login)

BeginInvoke
(Bất đồng bộ)



Không chặn luồng,
phù hợp cho cập
nhật liên tục
(e.g., Timer, Board)

TÓM TẮT SỰ KIỆN & HÀNH ĐỘNG UI

Sự Kiện (Event)	Loại (Category)	Hành Động UI
OnLoginResponse	Out-game	Chuyển từ Form Login sang Home
OnChallengeReceived	Out-game	Hiện Popup lời mời thách đấu
OnMoveNotify	In-game	Vẽ quân cờ lên bàn và đổi lượt
OnTimerUpdated	In-game	Cập nhật số giây đếm ngược trên label
OnGameEnded	In-game	Hiện thông báo kết quả, đóng GameBoard

Cập nhật giao diện (UI Update)

Out-game Events
(Invoke - Đồng bộ)

Log/Login, Thách đấu, Danh
sách, Lịch sử (đảm bảo thứ tự)

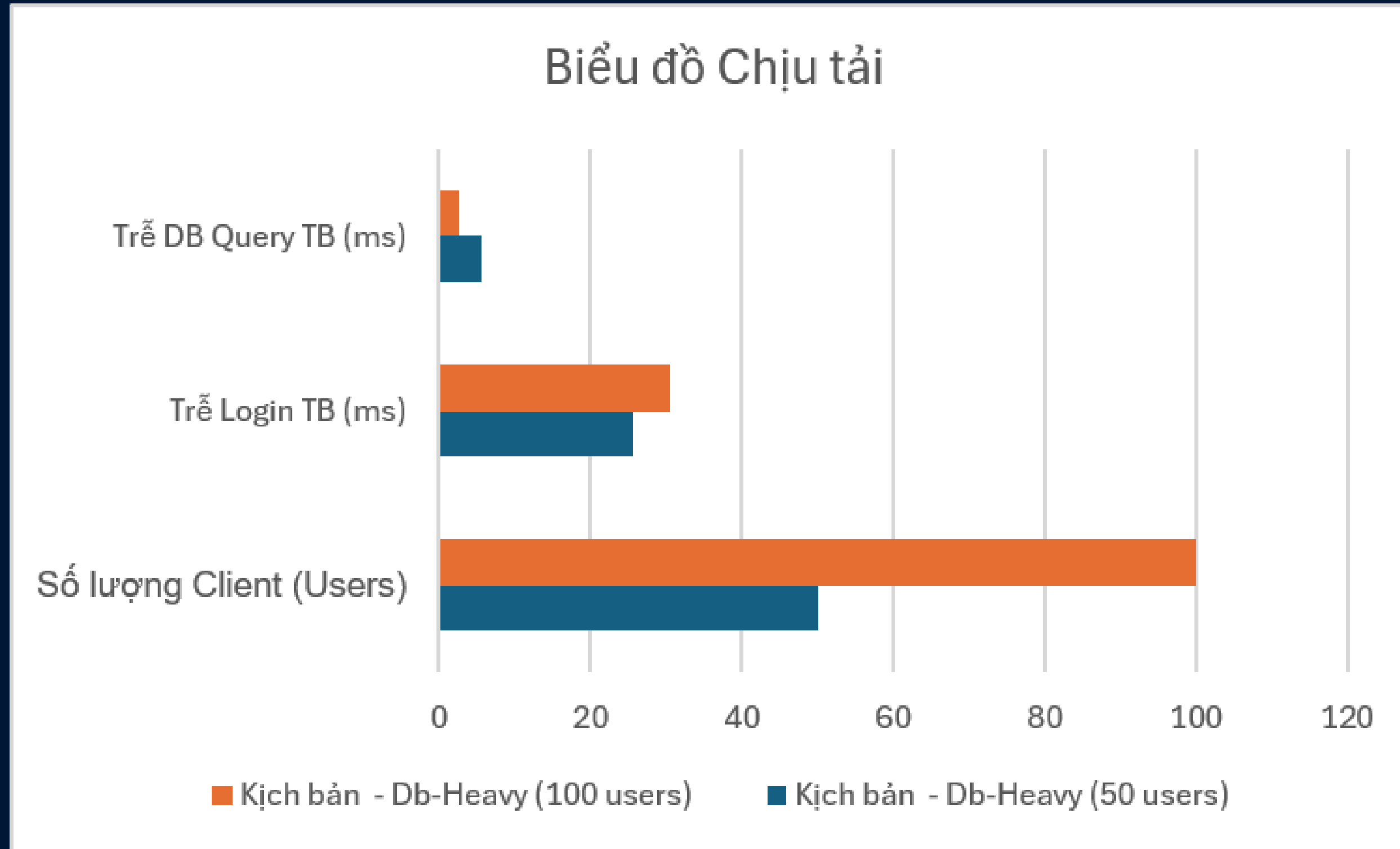


In-game Events
(BeginInvoke - Bất đồng bộ)

Nước đi, Đồng hồ, Chat, Kết thúc
(không chặn luồng)



BIỂU ĐỘ CHỊU TẢI



THANK YOU

